

DesPescar

Documentación de cambios y puesta en marcha del entorno local

Backend: MelinaCora/DesPescar , rama features/full-integration-docker-setup · Frontend: TenSF25/DesPescar , rama features/merge-koi-gateway · 4 de octubre de 2026

Cómo usar este documento. Describe los cambios hechos al poner en marcha el entorno local y el estado en que quedó. Para **levantar el entorno paso a paso** usá docs/INSTRUCTIVO-DOCKER.md . Algunas partes se actualizaron al publicarlo en el repositorio: el Docker vive en database/ y la contraseña de MySQL pasó a ser común.

1. Resumen

Se dejó funcionando el entorno local completo: base de datos MySQL en Docker, siete microservicios Spring Boot y el gateway, con datos de ejemplo y usuarios de prueba. Además se corrigieron dos problemas de comportamiento:

- El buscador de vuelos exigía iniciar sesión. Ahora la consulta de vuelos es pública y solo se pide login al avanzar con la compra.
- La redirección al login al seleccionar un vuelo fallaba y, tras iniciar sesión, no devolvía al usuario al vuelo elegido.

Alcance de la verificación. Los endpoints se probaron con curl contra el gateway. De los cambios en el frontend, el usuario confirmó que la redirección al login funciona. Que, tras iniciar sesión, se vuelva a /booking/baggage con el vuelo elegido **no fue confirmado en el navegador**.

2. Base de datos (MySQL en Docker)

Los servicios esperan MySQL en localhost:3306 con usuario root . En el repositorio, la carpeta database/ contiene todo lo necesario:

Archivo	Función
database/docker-compose.yml	MySQL 8 como despescar-mysql (puerto 3306, volumen mysql_data , healthcheck por TCP) y Adminer en el puerto 8090 .
database/init/01_create_databases.sql	Crea las 7 bases de datos la primera vez que arranca el contenedor.
database/.env.example	Plantilla de DB_PASSWORD . Normalmente no hace falta crear un .env .
database/seed/	Script para cargar datos de ejemplo (sección 4).
database/README.md	Guía de conceptos e instalación de Docker (autoría de Noelia, actualizada).

Bases de datos: despescar_identity , despescar_hotel , despescar_flight , despescar_package , despescar_reservation , despescar_payment y despescar_koiia . Todos los servicios usan ddl-auto=update , por lo que Hibernate crea las tablas al arrancar.

Contraseña común de MySQL

Antes cada persona escribía su propia contraseña en los siete `application.properties`, lo que producía conflictos constantes y dejaba contraseñas publicadas. Ahora los siete leen `spring.datasource.password=${DB_PASSWORD:despescar_dev}` y el compose usa el mismo valor por defecto, `despescar_dev`. Nadie tiene que tocar archivos. Los tests usan H2 en memoria y no se modificaron.

Atención. Quien ya tenga un volumen de MySQL creado con otra contraseña debe recrearlo (`docker compose down -v`), porque MySQL solo toma la contraseña al crear el volumen. Las contraseñas anteriores siguen en el historial de otras ramas del repositorio público; si se reutilizan en otro lado conviene cambiarlas.

Puerto de Adminer

El Adminer original usaba el puerto 8080, el mismo que `identity-service`, y provocaba el error *Port 8080 was already in use*. Ahora está en **`http://localhost:8090`** (servidor `mysql`, usuario `root`).

3. Servicios y puertos

Servicio	Puerto	Notas
<code>identity-service</code>	8080	Usuarios, login y roles. Crea 4 roles y 8 permisos al arrancar.
<code>flightservice</code>	8081	Aeropuertos, aerolíneas, vuelos y tarifas.
<code>hotel-service</code>	8083	Hoteles (tiene un <code>JWT_SECRET</code> por defecto).
<code>payment-service</code>	8084	No se arrancó. Pendiente.
<code>reservation-service</code>	8085	Reservas y asientos.
<code>package-service</code>	8086	Paquetes turísticos.
<code>gateway-service</code>	8087	Punto de entrada del frontend.
<code>koi-ia-service</code>	8088	Chatbot. Sin IA real: ver sección 8.

Cómo arrancar cada servicio

Cada servicio va en su propia terminal. Se usó `mvn` (Maven 3.8.7 instalado), porque los `mvnw` no tienen permiso de ejecución y `koi-ia-service` y `gateway-service` no traen `mvnw`.

```
export JWT_SECRET='despescar-dev-secret-key-2026-must-be-long-enough'
cd services/identity-service
mvn spring-boot:run
```

Variables de entorno que usa el proyecto:

Variable	Quién la necesita	Obligatoria
----------	-------------------	-------------

JWT_SECRET	identity, flight, reservation, package, payment, gateway	Sí, la misma en todos . Se usó una clave de ejemplo solo para desarrollo.
RESERVATION_SERVICE_SYNC_TOKEN	reservation-service y payment-service	Imprescindible para pagar, el mismo valor en los dos (ver 6.1). Si queda vacía los servicios arrancan, pero crear un pago falla con un error 502 ("Reservation-Service rechazó la consulta de la reserva").
GROQ_API_KEY	koi-ia-service	Sí para que responda la IA. Se arrancó con una clave de relleno.
MERCADOPAGO_*	payment-service	No para arrancar (valor vacío por defecto). <code>MERCADOPAGO_ACCESS_TOKEN</code> con una credencial de prueba es necesaria para generar el enlace de pago.

`koi-ia-service` **debe arrancarse así**, porque su código de pruebas no compila:

```
mvn -Dmaven.test.skip=true spring-boot:run
```

El archivo `KoiAiAssistantTest.java` usa `OllamaClient` y un método `extractTravelInfo` que ya no coinciden con el código actual (ahora usa Groq). No se modificó.

Límite de espera del gateway

El gateway corta cualquier petición a los 5 segundos con un error 504. Con el equipo sin memoria libre, el registro de usuarios tardaba más: el usuario se creaba en el backend pero el frontend recibía un error. Para esta ejecución se subió el límite a 30 segundos mediante un argumento de arranque, **sin modificar el `application.yml`**:

```
cd services/gateway-service
mvn spring-boot:run \
  -Dspring-boot.run.arguments="--
resilience4j.timelimiter.instances.gatewayCircuitBreaker.timeoutDuration=30s \
--spring.cloud.gateway.httpclient.response-timeout=30s"
```

Se suben **los dos** límites (el del circuit breaker y el de respuesta del cliente HTTP). Probado con un servidor de prueba que tarda 8 segundos: pasa por el gateway con 200; con el límite de 5 segundos cortaba con 504. Es necesario para el chatbot, que puede tardar hasta unos 10 segundos. Guía del chatbot: `docs/INSTRUCTIVO-CHATBOT-GROQ.md`.

4. Datos de ejemplo cargados

El proyecto no trae datos iniciales para vuelos, hoteles ni paquetes. Se cargaron por la API con un token de `SUPER_ADMIN`. Esa carga ahora está en el repositorio: `database/seed/seed.sh` (repetible, no duplica datos). Sus fechas son relativas al día en que se ejecuta; las de esta tabla son las de la primera carga manual.

Dato	Cantidad	Detalle
------	----------	---------

Aeropuertos	9	EZE, AEP, COR, MDZ, BRC, SLA, SCL, MIA, MAD
Aerolíneas	3	Aerolíneas Argentinas (AR), LATAM (LA), Flybondi (FO)
Tarifas	2	Light y Standard, asignadas a todos los vuelos
Vuelos	54	9 rutas con ida y vuelta, salidas el 20, 23 y 27 de octubre de 2026
Hoteles	6	Buenos Aires, Córdoba, Bariloche, Mendoza, Madrid y Miami
Paquetes	5 activos	Córdoba, Bariloche, Mendoza, Madrid y Miami

Rutas disponibles: EZE-COR, AEP-COR, AEP-BRC (Aerolíneas Argentinas y Flybondi), AEP-MDZ, AEP-SLA, EZE-SCL, EZE-MIA y EZE-MAD, cada una con su ruta inversa. Solo existen vuelos para esos pares exactos de aeropuertos y esas tres fechas.

Notas. Quedó un paquete de prueba llamado *Prueba* desactivado (`active=false`) porque el `DELETE` solo desactiva. Los datos viven en el volumen de Docker: si se borra, se vuelven a cargar con `database/seed/seed.sh`. En la carga manual original el paquete de Miami quedó enlazado por error al vuelo LA1026 (Santiago a Buenos Aires); el script lo corrige buscando el número de vuelo real de cada ruta.

5. Usuarios de prueba

Correo	Contraseña	Rol	Uso
admin@despescar.com	Admin2026!	SUPER_ADMIN	Administración y carga de datos
cliente@despescar.com	Cliente123!	USER	Probar la compra como cliente
usuario.prueba@despescar.com otro.prueba@despescar.com	Prueba123!	USER	Pruebas de registro

El administrador se registró por la API y se le asignó el rol con un `INSERT` en la tabla `user_roles` (`role_id = 2`), porque asignar roles ya exige ser `SUPER_ADMIN`. Existe también `noelia@test.com`, creado por el propio usuario. Son credenciales solo para desarrollo local.

6. Cambios en el código

6.1 Backend: consulta pública de vuelos (gateway)

Problema. El gateway exigía token Bearer en todo `/api/`, incluso para listar aeropuertos o buscar vuelos, por lo que el buscador aparecía vacío sin sesión. Además `flightservice` ya declaraba esas rutas como públicas, de modo que el bloqueo estaba solo en el gateway.

Cambio. Archivo `services/gateway-service/.../filter/GatewayJwtAuthFilter.java`: se agregó el método `isPublicFlightRead` y se invoca junto a `isPublicPath`. Deja pasar sin token únicamente peticiones `GET` a:

- `/api/flights`, `/api/flights/search` y `/api/flights/{id}`
- `/api/airports` y `/api/airports/code/{code}`
- `/api/fares`

Verificado con curl, sin token: aeropuertos, vuelos y búsqueda devuelven 200. Siguen protegidos: `POST` a `/api/airports` (401), `/api/airlines` (401), `/api/packages` (401),

`/api/users` (401) y `/hoteles` (403, lo responde el propio hotel-service).

La búsqueda exige los parámetros `origin`, `destination`, `departureDate` y `passengers`; `returnDate` es opcional.

Rutas internas bloqueadas. En esta rama el gateway además responde **403** a todo `/api/bookings/internal/**`. Son las rutas con las que `payment-service` consulta y confirma reservas (usan el encabezado `X-Internal-Service-Token`, con el valor de `RESERVATION_SERVICE_SYNC_TOKEN`). Antes solo las protegía ese token, y el gateway no las filtraba: un usuario con sesión podía llegar a ellas. Los servicios se hablan directo por su puerto, así que el flujo de pago no se ve afectado. Probado con el gateway levantado: las rutas internas dan 403 (con y sin token), mientras `/api/bookings/1`, `/api/payments` y `/api/users` siguen dando 401 sin sesión.

6.2 Frontend: redirección al login y retorno al flujo de compra

Problema. `ProtectedRoute` guardaba el `location` global del navegador, que no se puede almacenar en el historial, y la navegación a `/login` fallaba. Además, al iniciar sesión se enviaba siempre al inicio del rol, ignorando la página que el usuario intentaba abrir.

`PublicRoute`, que envuelve `/login`, también redirigía al inicio apenas el usuario quedaba autenticado.

Archivo (en <code>src/</code>)	Cambio
<code>components/Routes/ProtectedRoute.tsx</code>	Usa <code>useLocation()</code> de React Router y guarda esa ubicación como <code>state.from</code> .
<code>features/auth/postLoginPath.ts</code> (nuevo)	Función <code>getPostLoginPath</code> : devuelve la ruta guardada en <code>state.from</code> (solo si empieza con <code>/</code>) o, si no hay, el inicio del rol.
<code>features/auth/hooks/useAuth.ts</code>	El login navega al destino calculado con <code>getPostLoginPath</code> , con <code>replace: true</code> .
<code>components/Routes/PublicRoute.tsx</code>	Usa el mismo destino, para no competir con la redirección del login.

Se comprobó que el proyecto compila (`tsc --noEmit`) y pasa el linter en las carpetas modificadas.

6.3 Base de la rama y compatibilidad con el frontend

La rama parte de `features/auditoria-y-chatbot`, no de `features/payment-service-update`. Esta última agrega un commit (`7d5fab00`, Checkout API con pago grupal por fracciones) que **reemplaza** `POST /api/payments` por `/api/v1/payments`, lo deja sin autenticación (`permitAll`) y no está enrutado en el gateway. El frontend `features/merge-koi-gateway` paga con `POST /api/payments` y no tiene pantalla ni código para el pago grupal en ninguna de sus 21 ramas, por lo que ese commit queda fuera hasta que el equipo defina el contrato.

Llamadas del frontend verificadas (leyendo el código) contra el backend de esta rama:

Frontend	Backend
Login, registro, renovar token, <code>/api/users/me</code>	identity-service (vía gateway)
Aeropuertos, búsqueda y detalle de vuelos	flightservice; públicas en el gateway

<code>/api/bookings/flights/{id}/seats</code> y <code>/seat-map</code> ; WebSocket <code>:8085/ws-despescar</code>	reservation-service
<code>POST /api/bookings/init</code> y <code>PUT /api/bookings/{id}/passengers</code>	reservation-service (mismos campos)
<code>POST /api/payments</code> con <code>{reservationId}</code> y redirección al <code>checkoutUrl</code>	payment-service
Chatbot: <code>/api/koi/sessions</code> y <code>/messages (message + history)</code>	koi-ia-service

Límite de la verificación: es lectura de código; el flujo completo no se probó en ejecución. El pago además requiere una credencial de prueba de MercadoPago. Los cambios de asientos que el frontend tiene sin commitear no se revisaron.

Flujo de selección de vuelo

1. Búsqueda en `/vuelos` (pública).
2. Al elegir vuelo: si la búsqueda es de ida y vuelta, el primer clic elige la ida y el segundo la vuelta; con solo ida, el primer clic basta.
3. Se navega a `/booking/baggage` (ruta protegida): sin sesión se redirige a `/login` y, tras iniciar sesión, se vuelve a esa misma dirección.
4. Elección de tarifa y equipaje, luego `/booking/seats` (asientos) y `/booking/checkout`.

Los dos clics en una búsqueda de ida y vuelta son el comportamiento original del proyecto, no un fallo.

7. Problemas encontrados y cómo se resolvieron

Síntoma	Causa	Solución
Docker no conectaba	El servicio estaba apagado	<code>sudo systemctl start docker</code> (acción del usuario)
Access denied en MySQL	Contenedor y volumen anteriores con otra contraseña	Se eliminaron y se recreó con el <code>compose</code>
Could not resolve placeholder 'JWT_SECRET'	Variable de entorno no definida	Exportar <code>JWT_SECRET</code> antes de arrancar
Port 8080 was already in use	Adminer del proyecto anterior	Adminer recreado en el puerto 8090
<code>./mvnw: Permission denied</code>	Sin permiso de ejecución	Se usó <code>mvn</code> directamente
Error de compilación en <code>koi-ia-service</code>	Test desactualizado (<code>OllamaClient</code>)	<code>-Dmaven.test.skip=true</code>
Selector de origen/destino vacío	Base vacía y gateway exigiendo token; además 504 puntuales	Datos de ejemplo, rutas públicas en el gateway y más memoria
El registro "no funcionaba"	504 del gateway a los 5 s; la memoria swap estaba llena	Límite del gateway a 30 s; liberar memoria
Tras un apagón, todo caído	Los procesos no sobreviven al reinicio	Se volvió a levantar Docker y los servicios

8. Pendientes y recomendaciones

- **Memoria:** el equipo tiene 6,9 GB de RAM y el swap llegó a estar lleno (5,4 de 5,5 GB). Conviene cerrar IntelliJ y detener `koi-ia-service` si no se usa. Cerrar IntelliJ no afecta a los servicios, que no dependen de él.
- **Chatbot:** `koi-ia-service` necesita una `GROQ_API_KEY` real (<https://console.groq.com/home>). Sin ella no responde con IA.
- **payment-service** (puerto 8084) no se arrancó. Los datos de MercadoPago quedarían vacíos, por lo que los pagos no funcionarían de verdad.
- **Checkout API / pago grupal:** sigue en la rama `features/payment-service-update`. Antes de integrarlo hay que acordar con su autora o autor si se mantiene `POST /api/payments`, cómo se enruta en el gateway y cómo se autentica, y que el frontend tenga una pantalla para usarlo.
- **Contraseñas en el repositorio:** resuelto para MySQL en esta rama (variable `DB_PASSWORD` con valor común). Siguen existiendo contraseñas anteriores en el historial de otras ramas públicas.
- **Variables de entorno:** `JWT_SECRET` y `GROQ_API_KEY` solo existen en los procesos arrancados; hay que exportarlas de nuevo tras cada reinicio.
- **Docker al iniciar:** `sudo systemctl enable docker` evita tener que arrancarlo a mano.
- **Tests de koi-ia-service:** actualizar `KoiAiAssistantTest.java` al cliente actual.

- `/api/fares` no tiene ruta en el gateway (404). No afecta a la búsqueda, porque las tarifas vienen dentro de cada vuelo; haría falta solo para una pantalla de administración de tarifas.
- **Precios:** en la búsqueda el vuelo EZE-COR apareció como `ARS 15.060` y no `60`. No se investigó el motivo de la conversión.
- **Hoteles y paquetes** siguen exigiendo sesión. Si deben verse sin iniciar sesión, hay que cambiar el gateway y los `SecurityConfig` de `hotel-service` y `package-service`.
- **Scripts de carga:** ya están en `database/seed/`.

9. Guía rápida para levantar todo

1. **Docker:** `sudo systemctl start docker`
2. **MySQL:** `cd database && docker compose up -d` y esperar a *healthy*.
3. **Servicios**, cada uno en su terminal, con `JWT_SECRET` exportado (y `RESERVATION_SERVICE_SYNC_TOKEN` igual en reservas y pagos, `GR00_API_KEY` si corresponde): `identity-service`, `flightservice`, `hotel-service`, `package-service`, `reservation-service`, `payment-service`, `koi-ia-service` (con `-Dmaven.test.skip=true`) y por último `gateway-service`.
4. **Datos de ejemplo:** `./database/seed/seed.sh`
5. **Frontend** (`features/merge-koi-gateway`): `pnpm dev` en `http://localhost:5173`, que apunta al gateway en el puerto 8087.

El detalle completo y los problemas comunes están en `docs/INSTRUCTIVO-DOCKER.md`. Arrancar todos los servicios a la vez con poca memoria los hace tardar mucho más; conviene hacerlo de a uno o dos.